

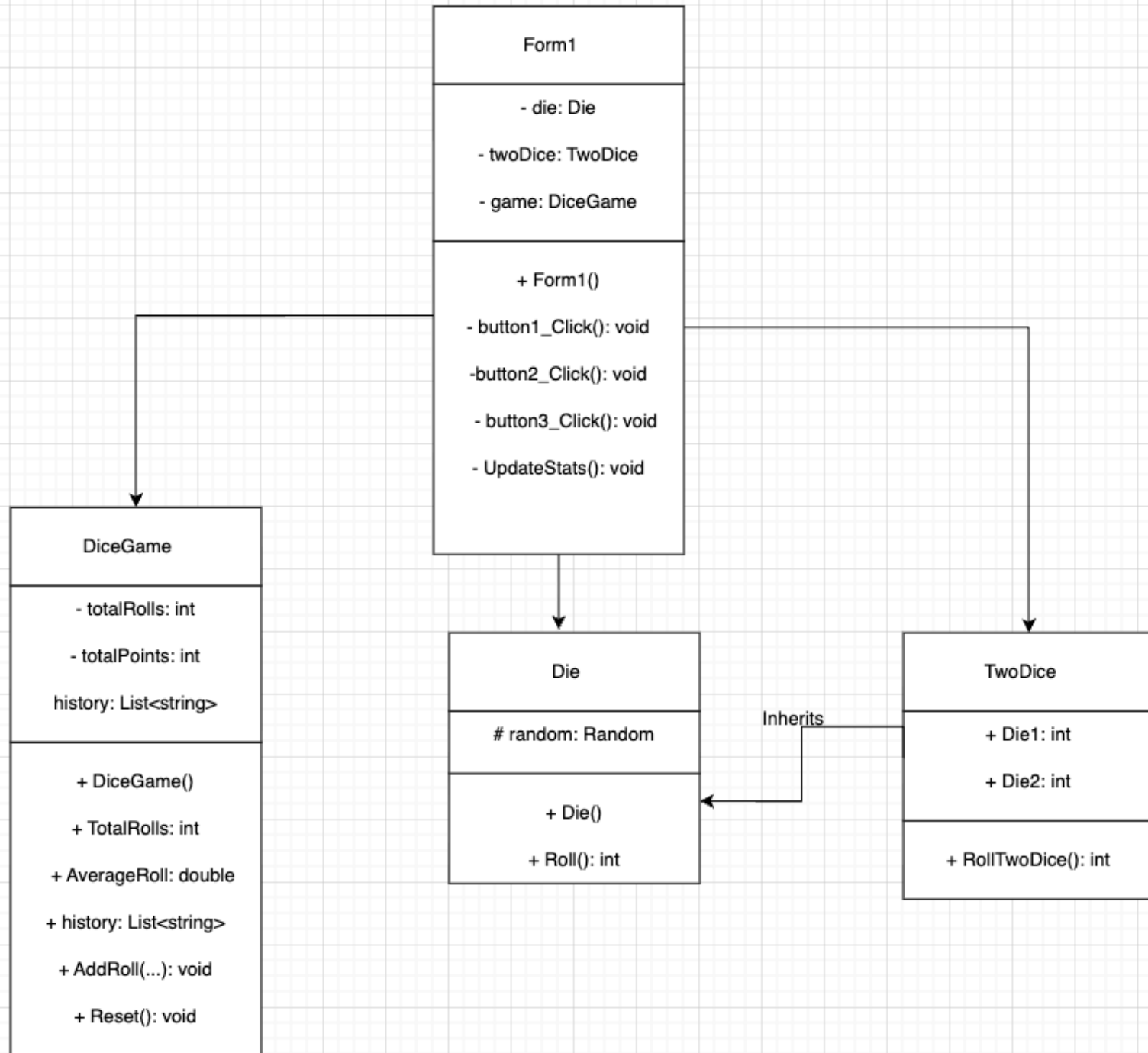
DICE ROLLING GAME



Dallon Adams

UML DIAGRAM

- Form1 controls the WinForms interface
- Die rolls a single 6 sided die
- TwoDie inherits from die class
- DiceGame tracks statistics
- Form1 uses other classes to update labels



CODE

```
//Inherits from the Die class
//TwoDice can reuse the Roll() method
2 references
internal class TwoDice : Die
{
    //stores the value of the first die after rolling
    4 references
    public int Die1 { get; private set; }
    //stores the second value of the second die
    4 references
    public int Die2 { get; private set; }
    //rolls the two diece and returns the total of the two
    1 reference
    public int RollTwoDice()
    {
        //inherieted roll method
        Die1 = Roll();
        Die2 = Roll();

        return Die1 + Die2;
    }
}
```

```
//updates the statistics after each roll
2 references
public void AddRoll(int total, string rollDescription)
{
    totalRolls++;
    totalPoints += total;
    history.Add("Roll " + totalRolls + ": " + rollDescription);
}
```

```
//This class is the logic for rolling a dice
4 references
internal class Die
{
    //Random funciton to generate number for dice roll
    //protected, so that child classes can use it
    protected Random random;

    //constructor for when a die object is created
    1 reference
    public Die()
    {
        random = new Random();
    }

    //Roll generates and returns a random number 1-6
    //virtual methodso child classes can overrride if necessary
    3 references
    public virtual int Roll()
    {
        return random.Next(1, 7);
    }
}
```

CODE

3 references

```
internal class DiceGame
{
    private int totalRolls;
    private int totalPoints;

    //Stores the history of each roll
    private List<string> history;
```

1 reference

```
public DiceGame()
{
    totalRolls = 0;
    totalPoints = 0;
    history = new List<string>();
}
```

1 reference

```
public int TotalRolls
{
    get { return totalRolls; }
}
```

```
public double AverageRoll
{
    get
    {
        if (totalRolls == 0)
        {
            return 0;
        }
        return (double)totalPoints / totalRolls;
    }
}
```

1 reference

```
public List<string> History
{
    get { return history; }
}
```

//updates the statistics after each roll

2 references

```
public void AddRoll(int total, string rollDescription)
{
    totalRolls++;
    totalPoints += total;
    history.Add("Roll " + totalRolls + ": " + rollDescription);
}
```

```
public void Reset()
{
    totalRolls = 0;
    totalPoints = 0;
    history.Clear();
}
```